Evaluation Report of the Urban Data Platform

Yixin Xia, Xinyue Feng, Hsi-Jui Chang

September 27, 2026

1 Evaluation Method

All measurements were taken on one MacBook Pro (Apple M5 Pro, 48 GB RAM) with Spark 4.2 in local mode (`local[*]`, 12 GB driver memory, 32 shuffle partitions) and Delta Lake 4.4. The evaluation code is in `src/data_intensive_computing/task5/` and is run with `uv run python -m data_intensive_computing.task5.cli`. Raw results are stored in `artifacts/task5_evaluation_results.jsonl`.

The update files were generated once with seed 42 and reused for every run. Before measuring, we recorded the current Delta version of every table under test. After each run, the tables were restored to these versions with Delta `RESTORE`. Every repetition therefore processed the same input against the same table state. The evaluation checked that every repetition inserted the same number of rows; a mismatch would stop the experiment.

Each experiment started with one discarded warm-up run, followed by five measured repetitions. The order of the configurations was rotated in every repetition, so no configuration always ran first. We report the median wall-clock time, measured outside the pipeline functions so that monitoring writes are included.

Table 1: Measured configurations

Config	Pipeline	Purpose
A1	Task 1, validation on, monitoring on	Incremental update time
A2	Task 1, validation on, monitoring off	A1 − A2 = monitoring overhead
A3	Task 1, validation off, monitoring off	A2 − A3 = validation overhead
B1	Task 2 incremental, monitoring on	Analytical refresh time
B2	Task 2 incremental, monitoring off	B1 − B2 = monitoring overhead
B3	Task 2 full recompute, no monitoring	Baseline for B2

Validation can only be switched off for the evaluation. We added an opt-in `validation_enabled` flag to `apply_incremental_update()`; its default is `True`, so the production pipeline is unchanged.

2 Incremental Update Performance

Table 2 shows the Task 1 results. One complete incremental update of all three datasets takes 24.2 s (range 23.3–24.5 s over five runs).

Table 2: Task 1 incremental update time (median of 5 runs, seconds)

Dataset	Input rows	A1 (production)	A2 (no monitoring)	A3 (no validation)
Taxi Trips	573,287	13.0	11.3	3.5
Weather	168	2.8	1.9	0.7
Air Quality	177,240	8.1	6.5	2.4
All three	750,695	24.2	19.8	6.8

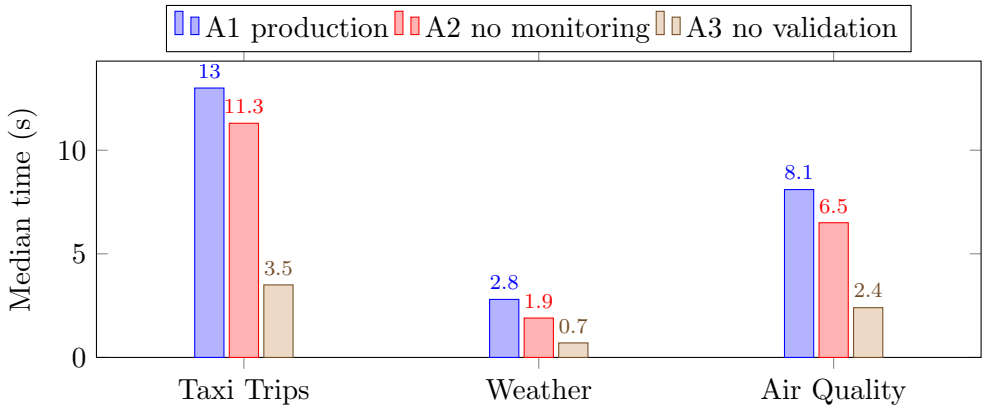


Figure 1: Task 1 time per dataset under the three configurations.

Time does not grow in proportion to input size. Weather has 168 rows but still takes 2.8 s, because every Delta read, validation step, and `MERGE` has a fixed cost that is independent of the number of rows.

3 Validation Statistics and Overhead

Table 3: Validation results for the generated release

Dataset	Input	Valid	Rejected	Rejection reason
Taxi Trips	573,287	477,739	95,548	`duplicate_existing_record`
Weather	168	168	0	–
Air Quality	177,240	177,240	0	–

All 95,548 intentional Taxi duplicates were detected and rejected, and no valid row was rejected. The accounting invariant `input` = `valid` + `rejected` held in every run.

Table 4: Validation overhead (A2 − A3)

Dataset	Extra time (s)	Relative to no validation
Taxi Trips	+7.8	+225%
Weather	+1.1	+162%
Air Quality	+4.1	+168%
All three	+13.0	+191%

Validation is the most expensive part of an incremental update: it accounts for 13.0 s of the 24.2 s production run (54%). The largest single cost is the duplicate check against the target table.

For Taxi Trips, the validator reads all existing `trip_id` values (about 9.5 million) and joins them with the new batch. The following Delta `MERGE` then compares the same keys against the same table again.

Without validation (A3), the pipeline still inserted exactly 477,739 Taxi trips, because the insert-only `MERGE` also skips existing keys. For this release, validation therefore did not change the stored data. Its value is protection against invalid values, missing references, and duplicates inside a batch, which this generated release does not contain. A release with invalid rows would also write more rows to the quarantine table, so the measured overhead is a lower bound for dirtier data.

4 Monitoring Results and Overhead

Each Task 1 execution writes three monitoring records (one per dataset). Each Task 2 execution writes seven (one for integration and one per analytical product). Every record is written with its own Delta transactions.

Table 5: Monitoring overhead

Pipeline	Records written	Extra time (s)	Per record (s)
Task 1 (A1 − A2)	3	+4.4 (+22%)	1.5
Task 2 (B1 − B2)	7	+7.3 (+26%)	1.0

The monitoring cost depends on the number of records written, not on the amount of data processed. Taxi Trips (573,287 rows) pays 1.7 s and Weather (168 rows) pays 0.9 s. For small datasets this is a large relative cost: monitoring adds 50% to the Weather update.

5 Analytical Refresh Performance

The 477,739 new Taxi trips fall into April, May, and June 2024. The incremental refresh (B2) integrates only these trips and recomputes only these three months. The full recompute (B3) re-integrates all trips and rebuilds all six products. B3 writes no monitoring records, so it is compared with B2.

Table 6: Task 2 analytical refresh (median of 5 runs, seconds)

Configuration	Median (s)	Range (s)
B1 incremental, monitoring on	35.4	32.5–36.5
B2 incremental, monitoring off	28.1	25.9–28.3
B3 full recompute	31.2	26.9–31.4

The incremental refresh is only 10% faster than a full recompute (28.1 s versus 31.2 s). With monitoring enabled, it is slower than the full recompute. Table 7 shows where the time goes.

Table 7: Task 2 time per step (median, seconds)

Step	Incremental (B1)	Full recompute (B3)
Integration	6.9	10.4
Airport transfer summary	2.8	1.4
Fare and tip patterns	2.5	1.2
Origin–destination flow	3.4	1.7
Trip distance distribution	2.5	1.2
Weather-sensitive zone demand	3.4	2.8
Zone hourly demand profile	4.2	1.8
Six products together	18.9	10.2

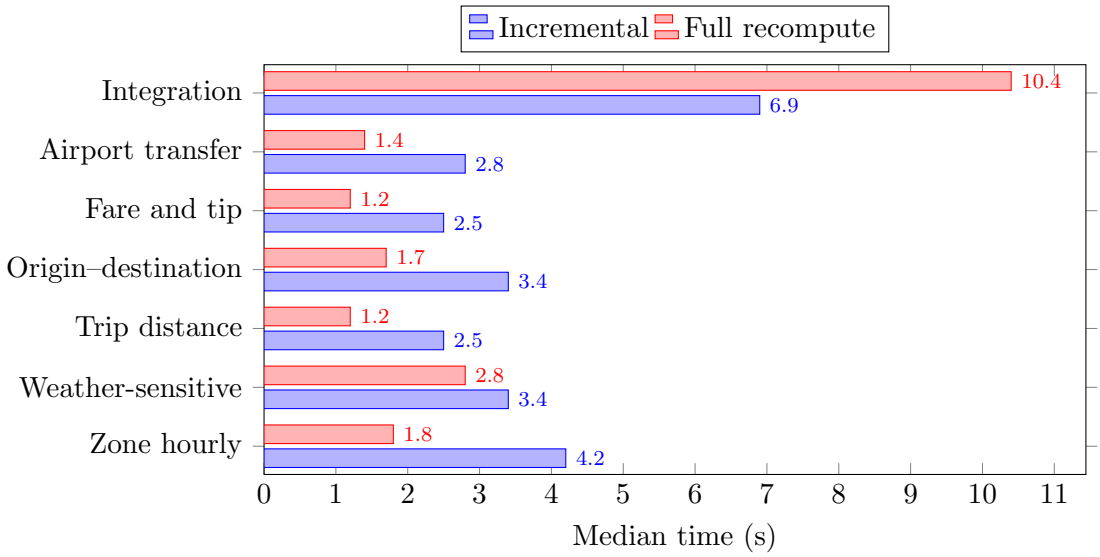


Figure 2: Integration is faster incrementally; every product is slower.

The integration step benefits from incremental processing: it enriches only the 477,739 new trips instead of all trips. Every analytical product, however, is slower incrementally. The incremental refresh runs one Delta `MERGE` per product and affected month, which is 18 merges in total. Each merge updates, inserts, and deletes rows, so Delta materialises the source and compares it with the target twice. The full recompute writes each product with a single overwrite. At this data size, the fixed cost of the 18 merges is larger than the work saved by processing only three months.

Issue found during the evaluation. The first evaluation run showed March 2024 as an affected month, although all new trips should start after the last original trip (2024-04-01 00:34:55). The update generator converted the zone-less TLC timestamps to epoch seconds as if they were UTC, while ingestion interprets them as New York local time. The first 1,385 generated trips were therefore four hours too early and landed on 2024-03-31. Because of these 0.3% of the trips, the incremental refresh recomputed all 3.6 million March trips and took 45.0 s instead of 35.4 s. We fixed the generator and repeated all measurements; this report uses the corrected run.

6 Storage Overhead

Storage was measured before and after one production run. We report the bytes added to disk and the size change of the current Delta version.

Table 8: Storage added by one incremental run

Table	Disk added (MB)	Current version added (MB)
Taxi Trips	45.7	45.3
Air Quality	14.0	13.9
Weather	0.0	0.0
Integrated Taxi Trips	50.7	50.3
Six analytical products	19.3	0.2
Quarantine (Taxi duplicates)	10.5	10.4
Monitoring tables	0.3	0.1

The processed and integrated tables grow in proportion to the new data (about 100 bytes per new trip in each table). The analytical products show write amplification: their current versions grew by only 0.2 MB, but 19.3 MB was written to disk, because Delta rewrites every file that contains an updated row and keeps the old files for time travel. The quarantine table stores the rejected duplicates again on every execution, so it grows even when the same release is applied twice.

Old files accumulate until `VACUUM` removes them. After the evaluation, which refreshed the integrated table more than 30 times (including ten full rebuilds) and restored it after each run, its directory held 5.7 GB of files that the current version no longer references, compared with a table size of about 1 GB. Every full rebuild leaves a complete old copy behind. A production deployment needs a scheduled `VACUUM`.

7 Maintainability

Week 1 decisions that simplified maintenance. Stable hash keys (`trip_id`, `observation_id`) made duplicate detection and idempotent insert-only merges straightforward. Delta Lake provided `MERGE`, additive schema evolution for `humidity` and `aqi`, and the `RESTORE` command that made this evaluation repeatable. Partitioning by pickup year and month matches the monthly slices that Task 2 refreshes. The Week 1 `ingestion_runs` table was extended by the monitoring system instead of being replaced, so earlier runs remained queryable.

Components that required the largest modifications. The Task 1 pipeline changed the most. Its original inline validation (about 200 lines) was replaced by a call to the shared Task 4 engine, and monitoring and evaluation hooks were added. The command-line interface was also affected: the Task 1 parser was copied into the Task 2 CLI to add new commands, so the original Task 1 CLI is no longer reachable and two copies must be kept in sync.

Support for future datasets. Adding a dataset mostly requires configuration: a `DatasetConfig` and a transform function for ingestion, and a `DatasetValidationConfig` built from reusable rules for validation. The validation engine does not need to change. Two limits remain. The primary key and required columns are defined separately in the Task 1 and Task 4 configurations. The analytical refresh is triggered only by new Taxi trips, so corrected Weather or Air Quality records do not refresh the products automatically.

What we would change in a redesign.

- Replace the per-month product merges with one partition overwrite per product (Delta `replaceWhere` on the affected months). This keeps the monthly recompute but removes the 18 merges that made the incremental product refresh slower than a full rebuild.
- Check duplicates against the target table only once: let the insert-only `MERGE` skip existing keys and take the duplicate count from its metrics, instead of scanning the target keys in the validator as well.

- Buffer monitoring records and write them in one transaction per pipeline execution instead of one per record.
- Use one dataset registry for keys, schemas, and rules instead of separate Task 1 and Task 4 configurations.
- Convert all timestamps to one explicit time-zone convention at the ingestion boundary and reuse the same helper in the update generator.
- Keep a single command-line parser for all tasks.

8 Scalability

The measured costs scale with different quantities:

- Validation scales with the size of the *existing* table, because every batch is compared against all stored keys. It will become slower as the platform accumulates history.
- Monitoring scales with the number of records written and is almost independent of data volume.
- The full recompute scales with the complete history. The incremental product refresh scales with the size of the affected months plus a fixed cost per merge. The incremental integration step also scans all stored trip keys to find pending trips.

With three months of history, one full recompute takes only 31 s on this machine, so fixed per-operation costs dominate and incremental refresh gains little. We expect the balance to change with more history: a full recompute must process every stored month, while the incremental refresh still processes only the affected months.

All results come from a single local machine and three months of Taxi data. They show relative costs between components; absolute times on a cluster would differ.